

TASK1

JDK

JDK是java的开发工具包，为程序员提供了编写，编译，调试Java程序所需要的工具，还包含了JRE以及基础类库。

JRE

JRE 为Java程序提供了一个运行环境，使得Java程序可以再不同的操作系统上运行。

JVM

JVM是Java虚拟机，在计算机上仿真模拟各种计算机功能来实现Java程序的跨操作系统运行，是执行Java程序的核心组件。

关系

- 1.JVM是JRE的核心组件，没有JVM,JRE无法执行Java程序
- 2.JDK包含了JRE，JRE包含了JVM
- 3.JDK用于开发，JRE用于运行，JVM为Java程序提供了一个平台去运行

TASK2

我配置的环境变量

新建了一个变量名叫JAVA_HOME

值为C:\Program Files\Java\jdk-1.8

再在Path中添加%JAVA_HOME%\bin

因为如果执行的程序在当前目录下不存在，Windows就会在一个叫做path的环境变量指定的目录中查找。配置环境变量就能保证无论在哪里打开终端都能执行javac，java 这两条指令

TASK3

1. .java文件是保存java代码的文件
2. 使用javac命令对java文件进行编译后就会生成.class文件，最终运行的时候是运行.class文件。

Plus Content

1. `.git` 目录是一个由 Git 版本控制系统管理的仓库中的重要组成部分.当使用 `git init` 命令时，就会在当前文件夹创建一个 `.git` 文件夹。这个目录包含了所有 Git 需要跟踪和管理仓库的信息。
2. git常用命令：
 1. **添加** `git add [file]` 将文件加入到Git暂存区，为提交做准备
 2. **提交** `git commit -m "输入日志的内容"` 将暂存区的改动正式提交到本地仓库的历史记录中，生成一个新的提交对象。

3. **回滚** `git reset` 用于撤销最近的提交或回到某个历史状态

`git revert` 用于创建一个新的提交来撤销之前的提交。

4. **签出** `git checkout [branch/tag/commit]` 切换到另一个分支、标签或提交，或者恢复工作目录中的文件。

5. **删除** `git rm [file]` 从仓库中删除指定的文件

6. **合并** `git merge [branch]` 将指定分支的变动合并到当前分支中。

7. **变基** `git rebase [branch]` 将当前分支的提交重新放置在另一个分支的顶部，通常用于将分支的提交顺序调整到最新的基础上。

8. **克隆** `git clone [repository URL]` 从远程仓库创建一个新的本地仓库副本。

9. **更新** `git pull` 用于从远程仓库拉取最新的提交，并与当前分支合并。

`git fetch` 仅下载远程分支的提交而不合并。

10. **将传入更改合并到当前分支** `git merge [remote/branch]` 将远程分支的最新提交合并到当前分支。

11. **在传入更改上变基当前分支** `git rebase [remote/branch]` 将当前分支的提交重新放置在远程分支的最新提交的基础上。

12. **推送** `git push [remote] [branch]` 将本地分支的提交推送到远程仓库。

3. **fork** 在GitHub等代码托管平台自己的账户下创建一个新的仓库放置项目的副本

`clone` 将一个远程仓库的所有分支和提交记录复制到本地作为一个副本，便于在本地开发和测试

`pull request` 将一个分支的变更合并到另一个分支，在提交变更之前，通过 `Pull Request` 可以让团队成员评审代码变更，并在确认无误后合并。

`push` 将本地仓库提交到远程仓库，通常是将本地的变更同步到远程仓库

4.四个区：

1. 工作区：就是一个编辑文件添加文件的文件夹
2. 暂存区：用 `git add [file]` 命令之后,将改动的文件添加到暂存区，准备进行提交
3. 本地仓库：`.git` 目录就是本地仓库。这个目录包含了所有提交的历史记录、分支信息等。
4. 远程仓库：远程仓库是位于服务器上的 Git 仓库，可以供多人协作使用。

5.文件移动：

1. `git add [file]` 命令之后,将改动的文件添加到暂存区
2. `git commit -m "message"` 将暂存区的文件提交到本地仓库
3. `git push [remote][branch]` 将本地仓库的某一个分支推送到远程仓库
4. `git pull [remote][branch]` 或者 `git fetch [remote][branch]` 从远程仓库拉取最新的提交，并与本地仓库合并或下载到本地。

6.冲突

当两个或多个开发人员修改了同一个文件的同一部分，并且这些修改不能自动合并时发生冲突。

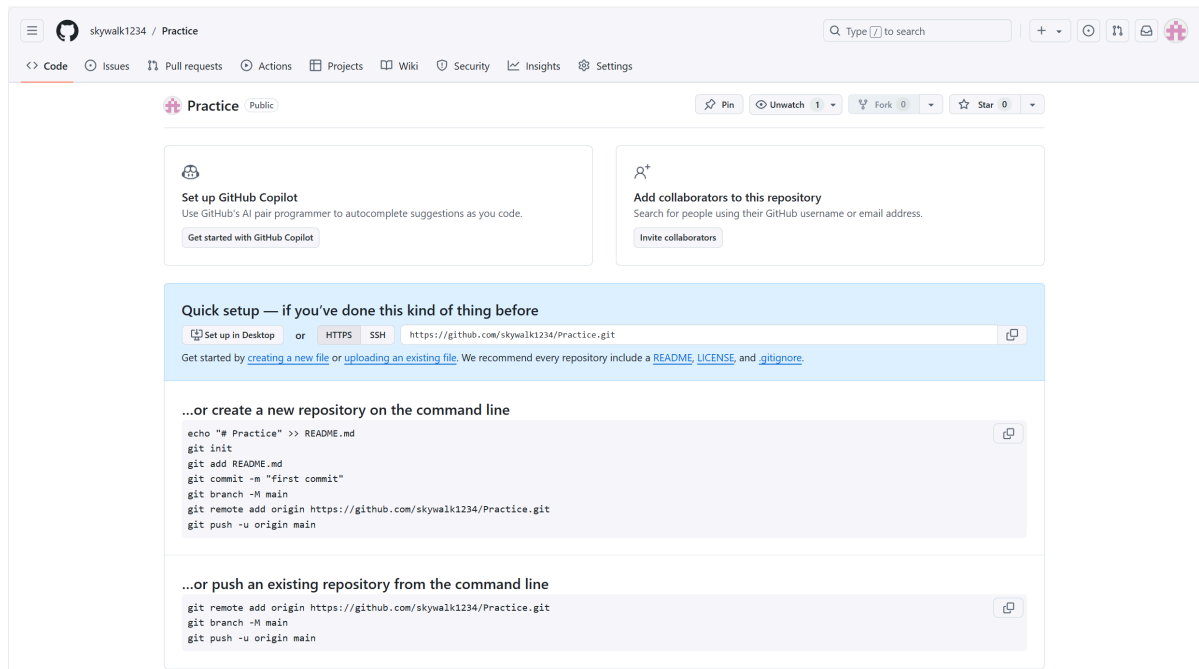
1. 合并 (merge)，拉取 (pull)，推送 (push)，Git在合并和拉取的时候存在冲突的文件会被标记为未合并，并在终端会有相应的提示

解决，打开标记为未合并的文件，手动删除冲突标记，并整合想要的部分

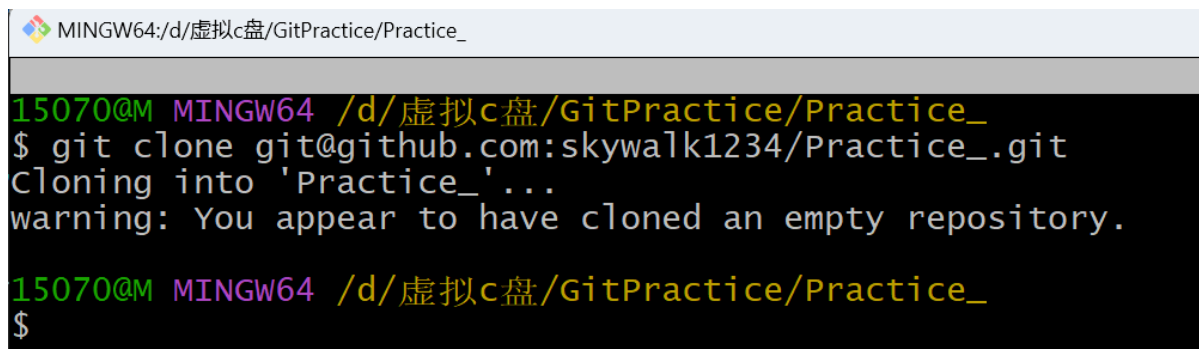
重新合并一次看是否成功；

7.实践过程

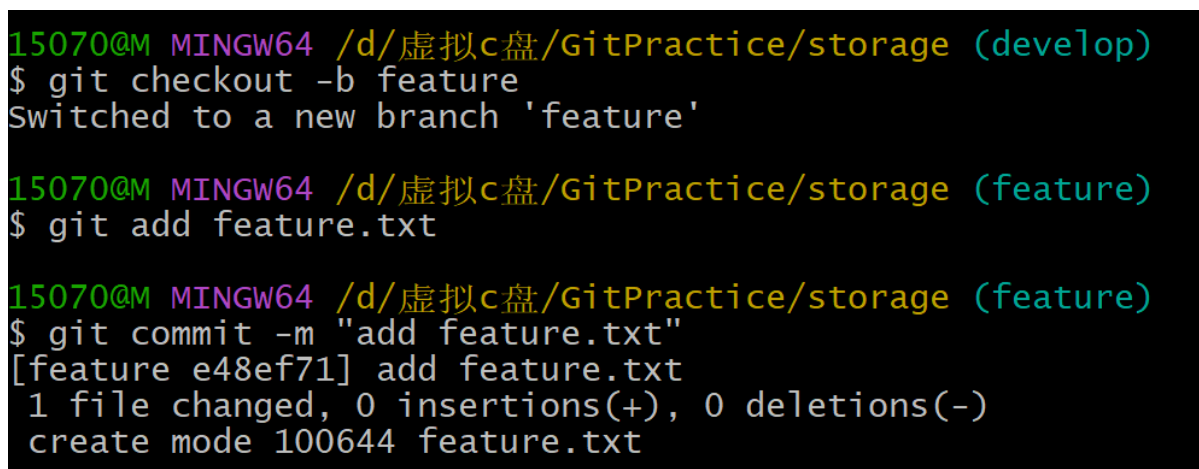
在GitHub上创建远程仓库



将远程仓库克隆到本地



新建feature分支，并新建一个feature.txt文件提交到本地仓库



合并feature分支到develop分支，并删除feature

```

15070@M MINGW64 /d/虚拟c盘/GitPractice/storage (feature)
$ git checkout develop
Switched to branch 'develop'

15070@M MINGW64 /d/虚拟c盘/GitPractice/storage (develop)
$ git merge feature
Updating 14fbc50..e48ef71
Fast-forward
 feature.txt | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 feature.txt

15070@M MINGW64 /d/虚拟c盘/GitPractice/storage (develop)
$ git branch -d feature
bash: git: command not found

15070@M MINGW64 /d/虚拟c盘/GitPractice/storage (develop)
$ git branch -d feature
Deleted branch feature (was e48ef71).

```

将develop合并到master并推送到远程仓库

Practice_ Public
Pin Unwatch

master 1 Branch Tags
 t
Add file <> Code

skywalk1234 add feautre 7fd4e74 · 28 minutes ago 2 Commits	
feature.txt	add feautre 28 minutes ago
hello.txt	add hello 31 minutes ago

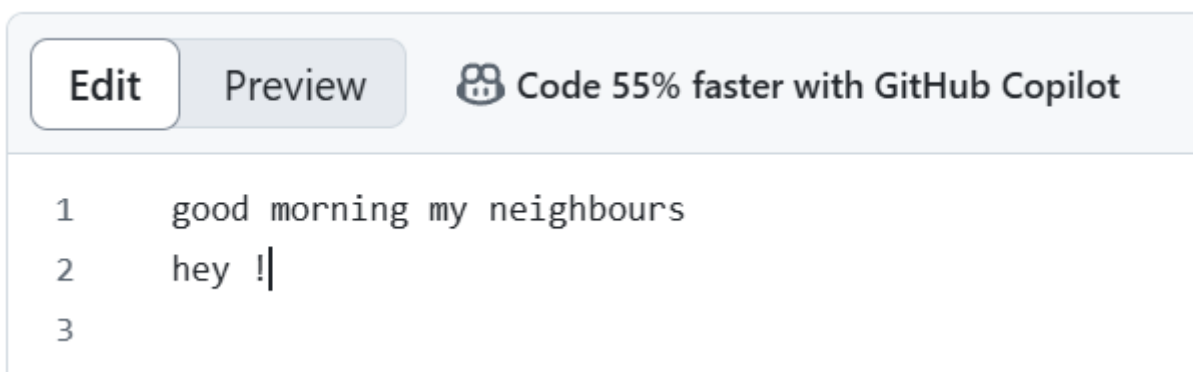
签出一个分支hot-fix，然后完成debug任务（创建一个debug.txt作为模拟），然后合并分支到master，并推送到远程仓库

Practice_ Public
Pin Unwatch 1

master 1 Branch Tags
 t
Add file <> Code

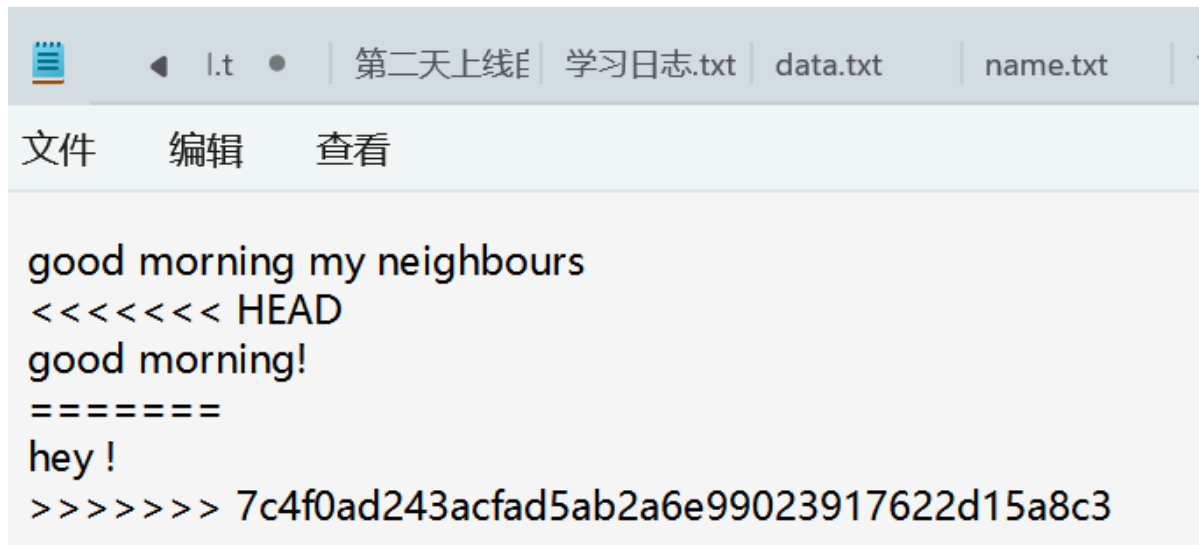
skywalk1234 add debug.txt 5784fc4 · 3 minutes ago 3 Commits	
debug.txt	add debug.txt 3 minutes ago
feature.txt	add feautre 37 minutes ago
hello.txt	add hello 40 minutes ago

对远程仓库的develop分支的hello.txt进行修改，模拟其他提交者的提交

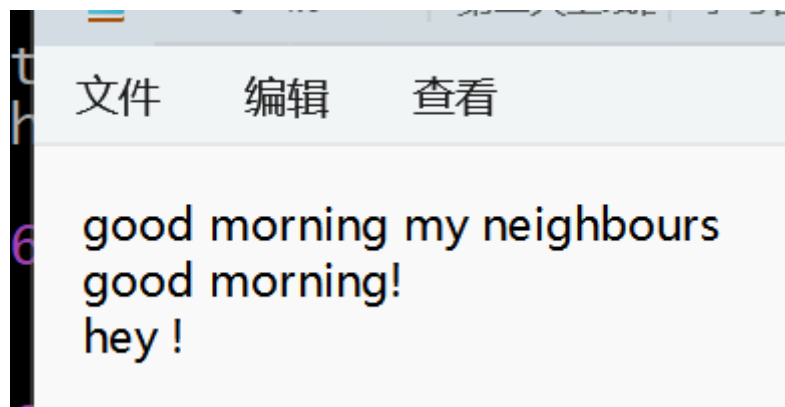


拒绝推送是因为远程分支的hello.txt文件被修改了，与本地推送上去的hello.txt不同，产生冲突。

当远程仓库和本地仓库的hello.txt文件都被修改了之后，再拉取远程仓库就会出现冲突



将那些标记删掉，并修改，再次推送到本地仓库，推送到远程仓库



Practice_ / hello.txt 

 skywalk1234 add new hello.txt

Code

Blame

4 lines (3 loc) · 52 Bytes

 Code 55% faster with GitHub Copilot

```
1    good morning my neighbours
2    good morning!
3    hey !
4
```

远程仓库成功显示最新的修改版本

多人协作开发

在GitHub上创建一个远程仓库，创建main分支作为项目的主分支。

每个开发的人要开发的时候拉取远程仓库以同步最新的版本，再checkout一个新的分支来开发。

开发完之后将分支合并回main分支并推送到远程仓库。如果产生冲突要即使沟通解决，确定最新的版本、

注意事项：

1. 要频繁拉取分支以及频繁推送分支，保证同步性
2. 在GitHub上设置分支保护规则，合并main分支前必须经过代码审查。
3. 代码写注释，确保别人能看懂
4. 使用版本控制标签来标记代码版本